Lecture 4: Linear Optimization

Zizhuo Wang

School of Data Science
The Chinese University of Hong Kong, Shenzhen

Jan 15, 2023

Logistics

- Homework 1 posted, due at noon (12pm), Feb 15th.

Recap: Linear Optimization

A *linear optimization* problem, or a *linear program* (LP) is an optimization problem in which the objective function and all constraint functions are linear (in the decision variables).

General form of linear program:

$$
\begin{aligned}
\text{minimize/maximize}_{\mathbf{x}} \quad & \mathbf{c}^T\mathbf{x} \\
\text{subject to} \quad & A_1\mathbf{x} \geq \mathbf{b}_1 \\
& A_2\mathbf{x} \leq \mathbf{b}_2 \\
& A_3\mathbf{x} = \mathbf{b}_3 \\
& x_i \geq 0 \qquad \forall i \in N_1 \\
& x_i \leq 0 \qquad \forall i \in N_2 \\
& x_i \text{ free} \qquad \forall i \in N_3
\end{aligned}
$$

Recap: Standard Form of Linear Optimization

An LP is said to be of *standard form* if it is of the form:

$$\begin{aligned} \text{minimize}_{\mathbf{x}} \quad & \mathbf{c}^T\mathbf{x} \\ \text{subject to} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq 0 \end{aligned}$$

where $\mathbf{x} \in \mathbb{R}^n$, A is an $m \times n$ matrix $(m < n)$ and $\mathbf{b} \in \mathbb{R}^m$.

We showed how to transform any linear program into a standard form. We also studied several types of problems that can be converted into linear optimization, including

- Minimax/maximin problems
- Problems with absolute values
- Linear fractional programming

Next, we show how to use software to solve optimization problem

Use Software to Solve LP

In this course, we mainly use MATLAB and Python:

- For MATLAB, we use a package called cvx
- For Python, we use a package called cvxpy
- Read the instruction documents
- `http://cvxr.com/cvx/`

You may also use Julia (cvx.jl).

Example: Production Planning Problem

$$
\begin{array}{lrrl}
\text{maximize} & x_1 & +2x_2 & \\
\text{subject to} & x_1 & & \leq 100 \\
& & 2x_2 & \leq 200 \\
& x_1 & +x_2 & \leq 150 \\
& x_1, & x_2 & \geq 0
\end{array}
$$

Example: Nurse Scheduling Problem

Let $d = [14, 15, 15, 16, 12, 6, 7]$, solve

$$
\begin{array}{lccccccccc}
\min & x_1 & +x_2 & +x_3 & +x_4 & +x_5 & +x_6 & +x_7 & & \\
\text{s.t.} & x_1 & & & +x_4 & +x_5 & +x_6 & +x_7 & \geq & d_1 \\
& x_1 & +x_2 & & & +x_5 & +x_6 & +x_7 & \geq & d_2 \\
& x_1 & +x_2 & +x_3 & & & +x_6 & +x_7 & \geq & d_3 \\
& x_1 & +x_2 & +x_3 & +x_4 & & & +x_7 & \geq & d_4 \\
& x_1 & +x_2 & +x_3 & +x_4 & +x_5 & & & \geq & d_5 \\
& & x_2 & +x_3 & +x_4 & +x_5 & +x_6 & & \geq & d_6 \\
& & & x_3 & +x_4 & +x_5 & +x_6 & +x_7 & \geq & d_7 \\
& x_1, & x_2, & x_3, & x_4, & x_5, & x_6, & x_7 & \geq & 0
\end{array}
$$

Example: Nurse Scheduling Problem

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{bmatrix} \quad \mathbf{c} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \quad A = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \quad \mathbf{d} = \begin{bmatrix} d_1 \\ d_2 \\ d_3 \\ d_4 \\ d_5 \\ d_6 \\ d_7 \end{bmatrix}$$

- The objective $\mathbf{c}^T\mathbf{x} = x_1 + x_2 + \cdots + x_7$.
- The constraint is $A\mathbf{x} \geq \mathbf{d}$, $\mathbf{x} \geq 0$

Example: Support Vector Machine Problem

$$
\begin{aligned}
\text{minimize}_{\mathbf{a},b,\delta,\sigma} \quad & \textstyle\sum_i \delta_i + \sum_j \sigma_j & \\
\text{subject to} \quad & \mathbf{x}_i^T \mathbf{a} + b + \delta_i \geq 1, & \forall i \\
& \mathbf{y}_j^T \mathbf{a} + b - \sigma_j \leq -1, & \forall j \\
& \delta_i \geq 0,\ \sigma_j \geq 0, & \forall i, j
\end{aligned}
$$

Example: Shortest Path Problem

Suppose we have a graph $G = (V, E)$, where $V = \{1, ..., n\}$ is the set of nodes and E is the set of edges.

- We denote the source node by 1, the terminal node by n
- We use w_{ij} to denote the distance from i to j. In general, w_{ij} does not necessarily equal w_{ji} (we call this a directed graph)
- We assume E contains all pairs of (directed) nodes: If there was no edge for (i, j), we can just set w_{ij} to be extremely large (larger than n times the maximum of the rest of w_{ij})

We want to write a general shortest path solver using LP:

- Input: A matrix $W = \{w_{ij}\}_{i,j=1,...,n}$
- Output: The shortest path from 1 to n and its distance

Example: Shortest Path Problem

We have derived the optimization formulation for this problem:

$$
\begin{aligned}
\text{minimize} \quad & \sum_{(i,j)\in E} w_{ij} x_{ij} \\
\text{subject to} \quad & \sum_j x_{sj} = 1 \\
& \sum_j x_{jt} = 1 \\
& \sum_j x_{ij} = \sum_j x_{ji}, \quad \forall i \neq s, t \\
& x_{ij} \in \{0, 1\}, \quad \forall (i,j) \in E
\end{aligned}
$$

Example: Shortest Path Problem

For simplicity of implementation, we further include x_{ii} as decision variables and set w_{ii} to be very large.

Decision variable: A matrix $X = \{x_{ij}\}_{i,j=1,\ldots,n}$

Objective function: $\sum_{i,j} w_{ij} x_{ij}$

- MATLAB representation: sum(sum(W .* X))

Constraints:

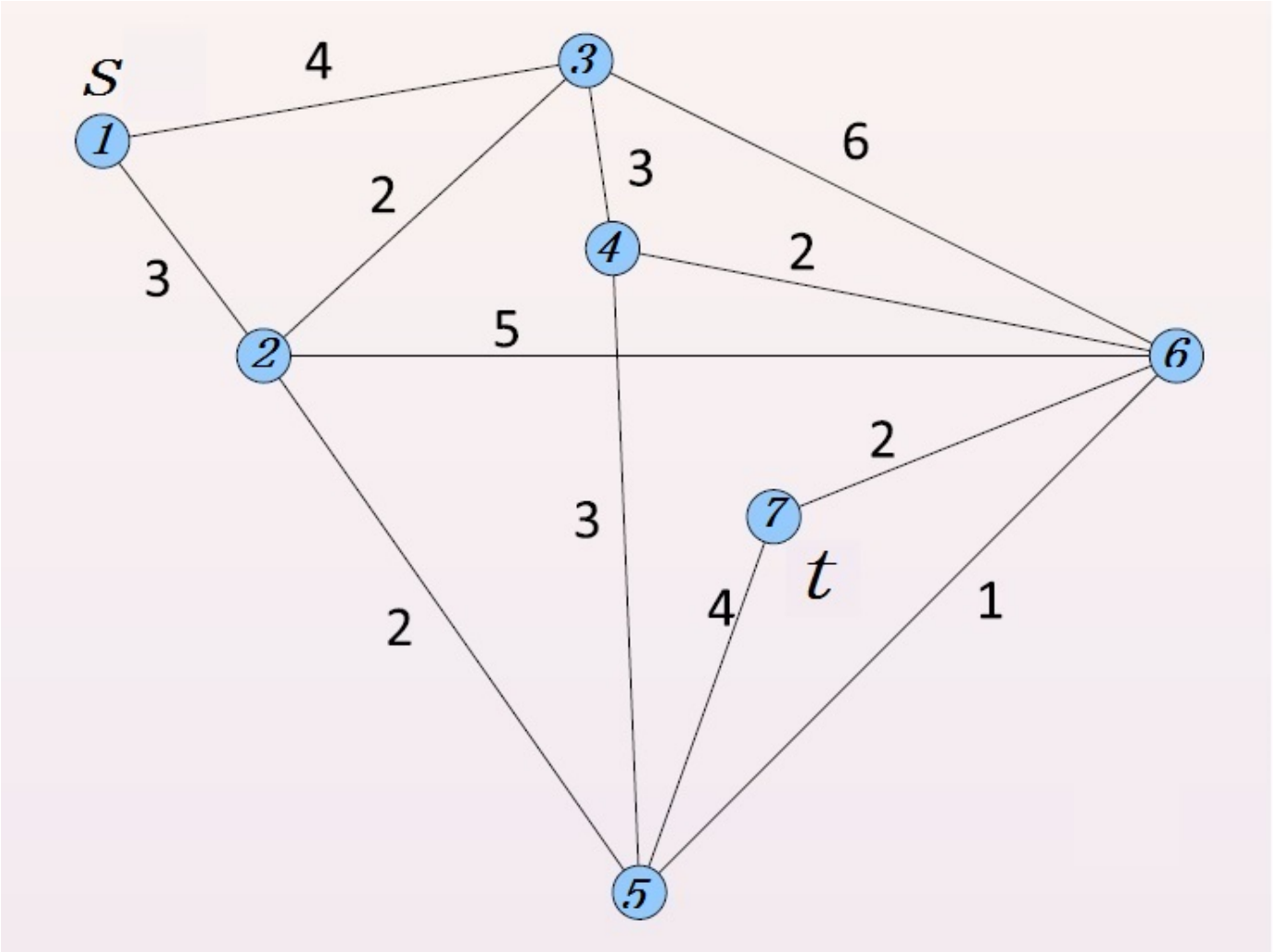
- sum$(X(1,:)) = 1$
- sum$(X(:,n)) = 1$
- sum$(X(i,:))$ - sum$(X(:,i)) = 0$ for $i \neq 1, n$

Integer constraints:

- We relax $x_{ij} \in \{0,1\}$ to $0 \leq x_{ij} \leq 1$. For this problem, this will not change the solution (will learn it later).

Example: Shortest Path Problem

After having the general solver, we can solve a specific problem:



Recap

Up to now, we have learned how to

- Formulate linear optimization problems
- Transform an LP into a standard form
- Use software to solve linear optimization

Next..

- How to solve linear optimization problem
- We will start with some basic properties of LP

Starting Point: Graphical Solutions to LP

It is very helpful to study a small LP from a graphical point of view.

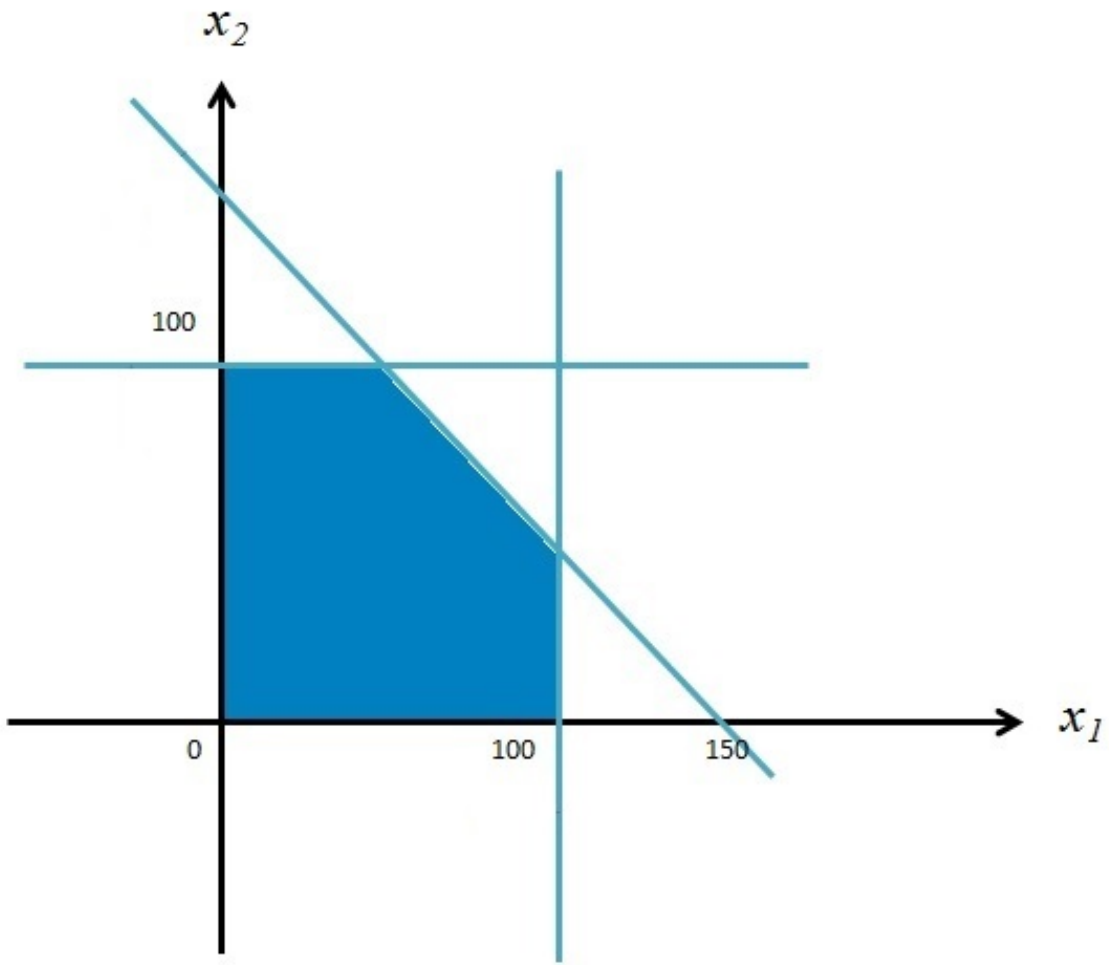
Recall the production problem:

$$
\begin{array}{lrrl}
\text{maximize} & x_1 & +2x_2 & \\
\text{subject to} & x_1 & & \leq 100 \\
& & 2x_2 & \leq 200 \\
& x_1 & +x_2 & \leq 150 \\
& x_1, & x_2 & \geq 0
\end{array}
$$

How can we solve this from a graph?

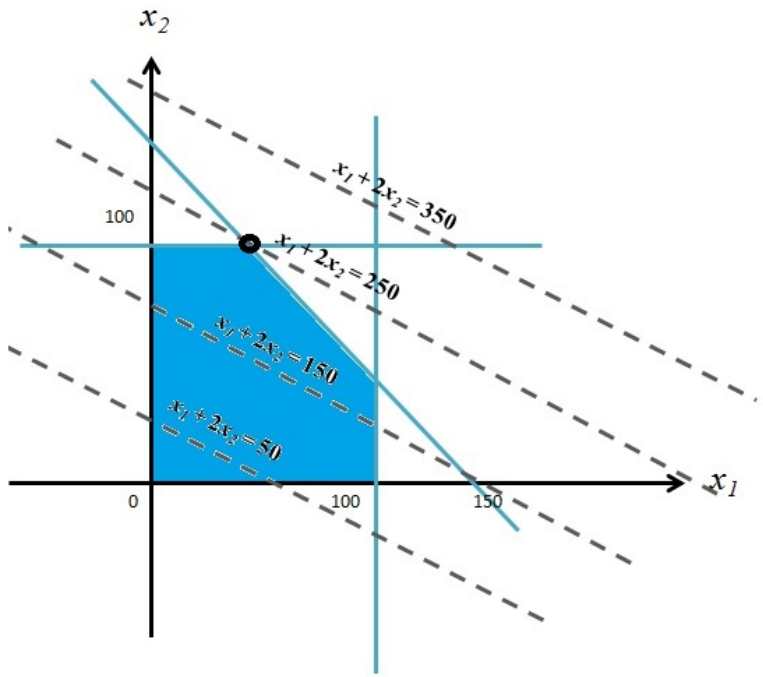
Solve LP from Graph

We first draw the feasible region.



To Maximize $x_1 + 2x_2$...

Then we draw the function $x_1 + 2x_2 = c$ for different values of c.



- The optimal solution is the highest one among these lines that touch the feasible region
- The coordinates: $(50, 100)$. Objective value: 250

Some Observations

- The feasible region of LP is a polygon
- The optimal solution tends to be at a corner of the feasible region
- Some constraints are *active* at the optimal solution ($x_2 \leq 100$, $x_1 + x_2 \leq 150$), some are not ($x_1 < 100$).

Next we will formalize these observations and study algorithms for solving LPs that can

- Guarantee to find the optimal solution
- Run within a certain (reasonable) amount of time

Some Definitions: Polyhedron

Definition

A *polyhedron* is a set that can be described in the form

$$\{\mathbf{x} \in \mathbb{R}^n | A\mathbf{x} \geq \mathbf{b}\}$$

where A is an $m \times n$ matrix and $\mathbf{b} \in \mathbb{R}^m$.

- Recall that in the standard form of LP, the feasible set is

$$A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq 0$$

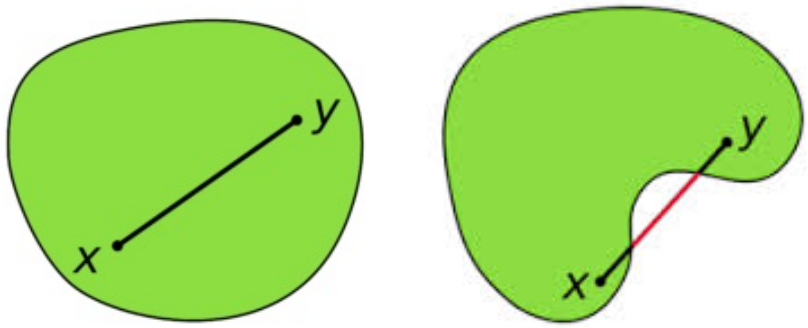
Is this a polyhedron? Why?

- Yes, we can write it as $A\mathbf{x} \geq \mathbf{b}$, $A\mathbf{x} \leq \mathbf{b}$, $\mathbf{I}\mathbf{x} \geq 0$ where $\mathbf{I}$ is the identity matrix.

Convex Sets and Convex Combinations

Definition

A set $S \subseteq \mathbb{R}^n$ is *convex* if for any $\mathbf{x}$, $\mathbf{y} \in S$, and any $\lambda \in [0, 1]$, $\lambda\mathbf{x} + (1 - \lambda)\mathbf{y} \in S$.



(a) Convex set (b) Non-convex set

Definition

For any $\mathbf{x}_1, ..., \mathbf{x}_n$ and $\lambda_1, ..., \lambda_n \geq 0$ satisfying $\lambda_1 + \cdots + \lambda_n = 1$, we call $\sum_{i=1}^{n} \lambda_i \mathbf{x}_i$ a convex combination of $\mathbf{x}_1, ..., \mathbf{x}_n$.

Extreme Points

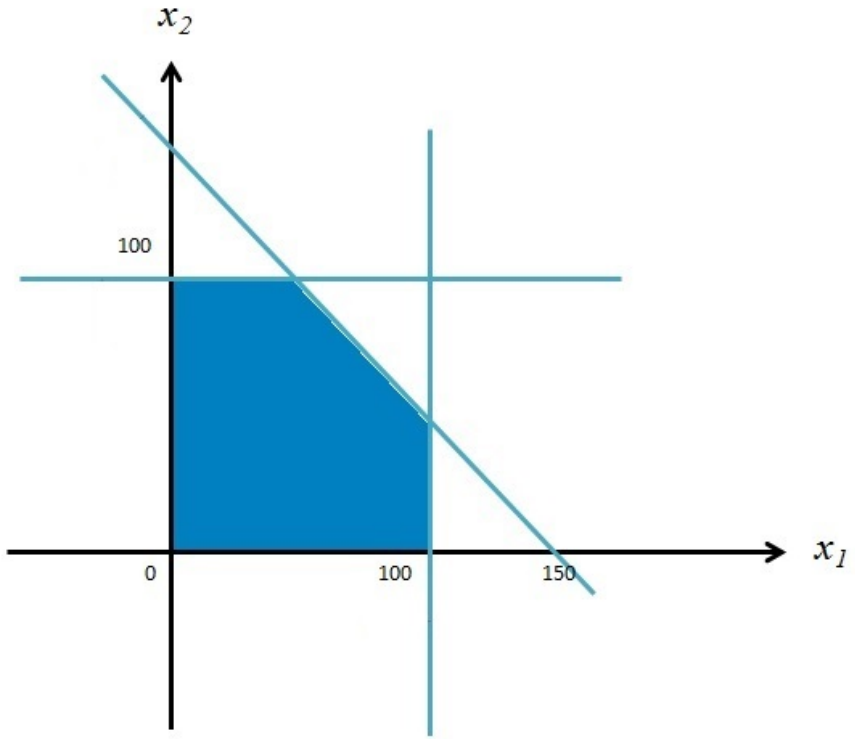
Remember we noticed that in an LP, the optimal solution tends to be on one of the corners of the feasible region. We first formalize this notion:

Definition

Let P be a polyhedron. A point $\mathbf{x} \in P$ is said to be an extreme point of P if we cannot find two vectors $\mathbf{y}, \mathbf{z} \in P$, both different from $\mathbf{x}$, and a scalar $\lambda \in [0, 1]$, such that $\mathbf{x} = \lambda \mathbf{y} + (1 - \lambda)\mathbf{z}$

- That is, $\mathbf{x}$ cannot be represented as a convex combination of other points in P.
- We sometimes call the extreme point the *vertex*, or *corner* of the polyhedron

Example



How many extreme points are there in this feasible region?

- Answer: 5

Extreme Points

We just gave the definition of the extreme points/vertices. However, this doesn't tell us how to find it. We want to have a good way of finding those points.

In the next lecture

- We will introduce an algebraic way to represent the extreme points
- We will show that it is sufficient to look among extreme points to solve a linear optimization problem
- Finally it will lead to an algorithm (the simplex algorithm) for solving LPs